Kubernetes Notes

Personal Reference

Last updated: March 11, 2026

Contents

---

1 Introduction

The purpose of this document is, obviously, my Kubernetes notes for myself. I am trying to avoid buzzwords and jargon, and explain both the *hows* and *whys* of Kubernetes.

That implies two basic questions worth answering before we go any further:

- What *is* Kubernetes?
- Why does Kubernetes exist – and why have so many people and organizations found it useful?

You'll often see Kubernetes described as "the operating system of the cloud." This is a buzzy way of describing it, but it is an accurate description. The different components of a Kubernetes cluster facilitate deploying applications in a cloud environment. That cloud may be public, like Amazon Web Services, or private, like your employer or school's server racks.

Software suited for running on a cloud should be durable and largely agnostic to what computer it runs on, aside from fundamental incompatibilities on the level of x86 vs. ARM processor architectures, Linux vs. Windows vs. macOS, and so on. As operating systems provide convenient interfaces to the physical computer programs run on, they (ideally!) remove a programmer's need to know the specific CPU, RAM, and networking details of the machine their code runs on. Kubernetes does the same one level up – abstracting away individual servers, nodes, regions, and storage types, handling concerns like application self-healing, upgrades and rollbacks, and storage on the programmer's behalf. This may not be a perfect analogy, but none is. It is the springboard for understanding those *hows* and *whys*.

Kubernetes was originally developed at Google, to make it easier for programmers to deploy and maintain programs at so-called "Google scale;" highly available, highly fault-tolerant systems. It is now owned and maintained by the Cloud Native Computing Foundation (CNCF). Before Kubernetes' release, containerization tools – Docker being the most known –

2 Components of a Kubernetes Cluster

A Kubernetes cluster consists of a **control plane node** (the "leader," "brain," or "head office" of the cluster) and one or more **worker nodes**. The control plane manages the desired state of the cluster; worker nodes run the actual workloads. Certain deployments, typically home lab or other sorts of non-production Kubernetes clusters, may run a single node which hosts both the control plane's components and runs workloads, but this is not advisable for production.

2.1 API Server

Coming soon!

2.2 `etcd`

A core component of Kubernetes is the durable `etcd` database. `etcd` is a key-value database (as opposed to a relational/SQL database, or a document store like MongoDB) used by many other distributed systems. `etcd` is a Kubernetes cluster's "source of truth" for the *desired cluster state*; a concept that is crucial to understanding and using Kubernetes.

Like most well-engineered pieces of software, access to the database is not a free-for-all affair. The Kubernetes API server is the intermediary between the `etcd` database and the outside world. The rest of this subsection discusses `etcd` and how Kubernetes uses it in greater depth; readers for whom this is not interesting or relevant can skip it.

High-availability Kubernetes servers may want to run multiple `etcd` instances; I will go into more detail on this later. Clusters using multiple `etcd`s use the *raft consensus algorithm* to designate a "leader" or main instance.

`etcd` consists of three executables:

1. `etcd` - the program running the database itself.
2. `etcdctl` - provides a command-line interface for querying the database and handling admin tasks
3. `etcdutl` - used for tasks that require operating directly on `etcd` data files, like migrating data between versions of `etcd`, restoring snapshots, defragmenting the database, and database validation

2.2.1 How Kubernetes stores data in `etcd`

`etcd` is a key-value database. Possible values for keys are seemingly pretty free-form, but Kubernetes uses a Linux path-like format for them. All keys start with `/registry/`, followed by the resource type, the namespace (if the object is namespaced), then the resource's name.

For example, the default namespace's definition is at `/registry/namespaces/default/`, and namespaced objects like Pods will live at, e.g. `/registry/pods/myAppNS/apiPod/`.

2.2.2 Using `etcdctl`

In most setups, `etcdctl` will require you to specify the API version, the endpoint and port the database server is listening at, and the certificate info. For a concrete example, if you're running a cluster locally using Docker Desktop, from within the Pod running `etcd`, your `etcdctl` commands may look something like

```
ETCDCTL_API=3 etcdctl get /registry/namespaces/default \
  --endpoints=https://127.0.0.1:2379 \
```

```
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key
```

The values for the Kubernetes objects are stored in Protobuf format, so while this command should run successfully (at least on a cluster created by Docker Desktop), the output will not be human-readable. There are multiple ways to turn the Protobuf data into more readable formats, but the primary one is, of course, the Kubernetes API.

2.2.3 Commands and Flags

Here are some `etcdctl` commands and flags worth knowing:

- `etcdctl get|put|del` - see, set, or delete entries.
- `etcdctl get -prefix` - get all items whose key starts with the given prefix.
- `etcdctl get -keys-only` - get only the key, not the value, if it exists. Useful in conjunction with `-prefix` to get every object of a specific type, in a specific namespace, or both.

Further reading

- https://etcd.io/docs/v3.6/

2.3 Controller Manager

Coming soon!

2.4 Scheduler

Coming soon!

2.5 Kubelet

Coming soon!

2.6 Kube-Proxy

Coming soon!